Les conteneurs à accès direct

Structures itérables

- En python, une structure de données ‘struct’ est dite itérable si on peut écrire une instruction de la forme :

```
for i in struct:
....
```

- Toutes les structures séquentielles sont itérables.

- Mais, il existe des structures itérables qui ne sont pas séquentielles :

  - Les ensembles
  - Les dictionnaires

Le type **ensemble** : Définition

- ensembles (set dans le language Python) sont des structures de données qui modélisent la notion mathématique d'ensemble.
- Un tel objet permet de créer des collections **non ordonnées** **mutables** constituées d'éléments **uniques**. Notez bien que les éléments ne sont pas ordonnés, donc non accessibles par des indices entiers, mais ils supportent l'itération.
- Chaque élément se présente une seule fois : les ensembles ne peuvent pas avoir plusieurs occurrences d'une même valeur.
- L'ordre des éléments d'un ensemble n'a pas d'importance.

Création des ensembles

Les ensembles peuvent être formés en utilisant l'une des méthodes suivantes:

```
In [42]: e1={'ticA','tekup',2022,4}

In [43]: e1
Out[43]: {2022, 4, 'tekup', 'ticA'}
```

1. En délimitant par des accolades une séquence de valeurs:

{val1, val2, val3, …, valn}

```
In [44]: e2={1,2-1,-1+2}

In [45]: e2
Out[45]: {1}
```

L'ordre des éléments dans une énumération ne se retrouve pas dans la réponse fournie par l'interpréteur.

les ensembles n'acceptent pas la répétition d'un même élément.

Création des ensembles

```
In [28]: e1=set()

In [29]: e1
Out[29]: set()

In [30]: type(e1)
Out[30]: set

In [31]: len(e1)
Out[31]: 0
```

ensemble vide

2. Construction avec set() :

La fonction *set* permet la construction d'un ensemble à partir de n'importe quelle valeur itérable. En voici quelques exemples :

```
In [46]: e3=set('tekup')

In [47]: e3
Out[47]: {'e', 'k', 'p', 't', 'u'}
```

```
In [48]: e4=set([1,2,3,4,5])

In [49]: e4
Out[49]: {1, 2, 3, 4, 5}
```

```
In [50]: e5=set(range(1,10,2))

In [51]: e5
Out[51]: {1, 3, 5, 7, 9}
```

Création des ensembles

Remarque :

A la différence des chaines, tuples et listes, l'ensemble vide ne peut pas être écrit littéralement par l'expression {} car on obtiendra un dictionnaire et non un ensemble :

```
In [58]: faux_ensemble={}

In [59]: type(faux_ensemble)
Out[59]: dict
```

→ **Il faut impérativement utiliser la fonction set()**

```
In [61]: vrai_ensemble=set()

In [62]: type(vrai_ensemble)
Out[62]: set
```

Création des ensembles

3. Construction en compréhension :

```
In [63]: e7={x*2 for x in range(10)}

In [64]: e7
Out[64]: {0, 2, 4, 6, 8, 10, 12, 14, 16, 18}
```

Création des ensembles

Remarque sur le contenu des ensembles :

À la différence des listes, un ensemble ne peut pas contenir tout type de données.

```
In [65]: e8={[1,2,5]}
Traceback (most recent call last):

  File "C:\Users\Sawssen\AppData\Local\Temp/ipykernel_11848/2857258313.py", line 1, in <module>
    e8={[1,2,5]}

TypeError: unhashable type: 'list'
```

on ne peut pas mettre une liste dans un ensemble

```
In [66]: e9={{1,8,5}}
Traceback (most recent call last):

  File "C:\Users\Sawssen\AppData\Local\Temp/ipykernel_11848/2835265942.py", line 1, in <module>
    e9={{1,8,5}}

TypeError: unhashable type: 'set'
```

on ne peut pas mettre d'ensemble dans un ensemble

→ **Un ensemble peut contenir:**

- **Tous les types simples (nombres entiers ou flottants, booléens)**
- **Des structures non mutables (chaînes, tuples)**

Parcours des éléments d'un ensemble

- L'absence de contrôle de l'ordre des éléments enlève l'intérêt de la notation indicielle

```
In [69]: e1
Out[69]: {2022, 4, 'tekup', 'ticA'}

In [70]: e1[0]
Traceback (most recent call last):

  File "C:\Users\Sawssen\AppData\Local\Temp/ipykernel_11848/1545461590.py", line 1, in <module>
    e1[0]

TypeError: 'set' object is not subscriptable
```

- En revanche, l'opérateur **in** permet de tester l'appartenance d'un élément à un ensemble.

```
In [71]: 'tekup' in e1
Out[71]: True

In [72]: 10 in e1
Out[72]: False
```

- Les ensembles sont des structures de données itérables

```
In [74]: e1
Out[74]: {2022, 4, 'tekup', 'ticA'}

In [75]: for i in e1: print(i)
ticA
tekup
4
2022
```

Opérations spécifiques aux objets set

Opération	Résultat
len(*ens*)	le cardinal de l'ensemble
elt in *ens*	True si l'élément est présent dans l'ensemble, False sinon
elt not in *ens*	True si l'élément est absent de l'ensemble, False sinon
ens.add(*elt*)	ajoute un élément à un ensemble
ens.remove(*elt*)	retire un élément à un ensemble (KeyError si élément absent)
ens.pop()	renvoie et supprime un élement arbitraire de l'ensemble (KeyError si ensemble vide)
ens.discard(*elt*)	retire un élément à un ensemble s'il y est effectivement (pas d'erreur sinon)
ens.clear()	efface le contenu de l'ensemble
ens.copy()	une copie de l'ensemble, indépendante de l'original

Opérations spécifiques aux objets set

Opération	Résultat
ens_1.isdisjoint(ens_2)	True si ens_1 et ens_2 sont disjoints
$ens_1 <= ens_2$ ens_1.issubset(ens_2)	True si ens_1 est inclus dans ens_2
$ens_1 < ens_1$	True si ens_1 est inclus strictement dans ens_2
$ens_1 > ens_2$ ens_1.issuperset(ens_2)	True si ens_1 contient ens_2
$ens_1 \mid ens_2$ ens_1.union(ens_2)	renvoie l'union des deux ensembles
ens_1 & ens_2 ens_1.intersection(ens_2)	renvoie l'intersection des deux ensembles
$ens_1 - ens_2$ ens_1.difference(ens_2)	renvoie la différence ensembliste
ens_1 ^ ens_2 ens_1.symetric_difference(ens_2)	renvoie la différence symétrique

Application

Définir deux ensembles (sets) : X = {a,b,c,d} et Y = {s,b,d}, puis **affichez** les résultats suivants :

I. – les ensembles initiaux ;

II. – le test d'appartenance de l'élément 'c' à X ;

III. – le test d'appartenance de l'élément 'a' à Y ;

IV. – la différence entre X et Y ;

V. – l'ensemble Z = X union Y ;

VI. – l'ensemble W = X inter Y.

```
In [90]: X=set('abcd')

In [91]: Y=set('sbd')

In [92]: print('X=',X)
X= {'a', 'b', 'c', 'd'}

In [93]: print('Y=',Y)
Y= {'b', 'd', 's'}

In [94]: print("appartenance de 'c' à X : ", 'c' in X)
appartenance de 'c' à X :  True

In [95]: print("appartenance de 'a' à Y : ", 'a' in Y)
appartenance de 'a' à Y :  False

In [96]: print('différence entre X et Y : "', X-Y)
différence entre X et Y : " {'a', 'c'}

In [97]: print('différence entre Y et X (Y-X) : "', Y-X)#les éléments de Y qui ne sont pas dans X
différence entre Y et X (Y-X) : " {'s'}

In [98]: print('union de X et Y', X|Y)
union de X et Y {'a', 'b', 'd', 'c', 's'}

In [99]: print('intersection de X et Y', X&Y)
intersection de X et Y {'b', 'd'}
```

Le type **Dictionnaire** : Définition

- Les *dictionnaires* sont des structures de données permettant **d'associer une *valeur* à une *clé*.**

- Exemple
```
l = [('banane', 2.59), ('pomme',1.95), ('orange',1.95)]
```

Cette liste associe un prix au kg à des fruits. Elle peut être considérée comme une **liste d'association** dans laquelle les fruits sont les ***clés*** et les prix les ***valeurs* associées**.

- Dans un dictionnaire, il ne peut y avoir qu'une seule association pour une clé donnée. Dit autrement, dans un dictionnaire les clés sont **uniques**.

Le type **Dictionnaire** : Définition

- Les dictionnaires sont des structures mutables **d'éléments hétérogènes**, **non ordonnées,** formées d'enregistrements du type ***clé:valeur***.

- Conteneur associatif : Associations (map) clé ⇒ valeur. Le seul moyen d'accéder à une valeur particulière est par l'intermédiaire de sa clé.

- Les clés doivent être **de type simple immutable** (on utilise le plus souvent des chaînes ou des entiers) et **uniques** mais pas nécessairement du même type.

Le type **Dictionnaire** : Construction

Les dictionnaires peuvent être formés en utilisant l'une des méthodes suivantes:

1. Construction littérale : obtenue par :

- énumération explicite des associations clé-valeur sous la forme ***clé : valeur***
- Les associations sont séparées par des virgules.
- Le tout entre accolades.

```
In [2]: tarifs={'banane':2.59, 'pomme':1.95,'orange':1.95}

In [3]: tarifs
Out[3]: {'banane': 2.59, 'pomme': 1.95, 'orange': 1.95}

In [4]: type(tarifs)
Out[4]: dict
```

Le type **Dictionnaire** : Construction

Les dictionnaires peuvent être formés en utilisant l'une des méthodes suivantes:

2. Construction avec la fonction dict() :

```
In [6]: tarifs
Out[6]: {'banane': 2.59, 'pomme': 1.95, 'orange': 1.95}

In [7]: tarifs2=dict([('banane',2.59),('pomme',1.95),('orange',1.95)])

In [8]: tarifs2
Out[8]: {'banane': 2.59, 'pomme': 1.95, 'orange': 1.95}

In [9]: type(tarifs2)
Out[9]: dict
```

Le type **Dictionnaire** : dictionnaire vide

Un dictionnaire qui ne contient aucune association est un dictionnaire *vide*

Pour obtenir un dictionnaire vide, on peut donner l'expression :

```
In [10]: d1={}

In [11]: d1
Out[11]: {}

In [12]: type(d1)
Out[12]: dict
```

Ou encore :

```
In [13]: d2=dict()

In [14]: d2
Out[14]: {}
```

Opérations spécifiques aux objets dict

Opération	Résultat
len(*dic*)	la longueur du dictionnaire (nombres de clés)
dic[*cle*]	la valeur associée à la clé (lève KeyError si la clé est absente)
dic.get(*cle*)	la valeur associée à la clé, et renvoie None si la clé est absente
dic.get(*cle*,*f*)	la valeur associée à la clé, et la valeur de *f* si la clé est absente
dic.setdefaut(*cle*)	la valeur si la clé est présente, sinon crée le couple cle :None
dic.setdefaut(*cle*,*f*)	la valeur si la clé est présente, sinon crée le couple *cle* :*f*
dic.items()	un itérable pour décrire les couples (clé,valeur) dans une boucle for
dic.keys()	un itérable pour décrire les clés du dictionnaire dans une boucle for
dic.values()	un itérable pour décrire les valeurs du dictionnaire dans une boucle for
dic[*cle*] = *val*	crée (ou modifie) une paire clé/valeur
dic.update(dic_2)	fusionne le dictionnaire dic_2 sur le dictionnaire *dic* (remplace les valeurs si clés homonymes)
cle in *dic*	True si la clé est présente dans le dictionnaire, False sinon
cle not in *dic*	True si la clé est absente dans le dictionnaire, False sinon
del *dic*[*cle*]	efface une paire clé/valeur
dic.clear()	efface le contenu du dictionnaire
dic.pop(*cle*)	renvoie la valeur, et supprime la paire clé/valeur (KeyError si clé absente)
dic.pop(*cle*,*f*)	renvoie la valeur, et supprime la paire clé/valeur (évalue *f* si clé absente)
dic.popitem()	renvoie une paire arbitraire (clé,valeur) et la supprime de dic (KeyError si dic vide)
dic.copy()	une copie du dictionnaire, indépendante de l'original

- dic_2 est un dictionnaire. Il peut aussi être une liste de listes [*cle*,*val*] ou de tuples (*cle*,*val*) ;
- *cle* une clé et *val* une valeur.

Le type **Dictionnaire** : les méthodes de parcours

1. *Parcours des clès*: itérer un dictionnaire, c'est itérer les clès:

```
for cle in d:
    print(d[cle])
```

```
In [10]: tarifs
Out[10]: {'banane': 2.59, 'pomme': 1.95, 'orange': 1.5}

In [11]: for cle in tarifs: print(tarifs[cle])
2.59
1.95
1.5
```

2. *Parcours des valeurs:*

```
for val in d.values():
    print(val)
```

```
In [9]: for val in tarifs.values(): print(val)
2.59
1.95
1.5
```

3. *Parcours des clès et valeurs simultanément:*

```
for cle,val in d.items():
    print('la clè {} contient la valeur {}'.format(cle,val))
```

```
In [14]: for cle,val in tarifs.items(): print ('la clé {} contient la valeur {}'.format(cle,val))
la clé banane contient la valeur 2.59
la clé pomme contient la valeur 1.95
la clé orange contient la valeur 1.5
```

Le type **Dictionnaire** : Application

Soit le dictionnaire :

```
>>> d = {'nom': 'Dupuis', 'prenom': 'Jacque', 'age': 30}
```

1. Corriger l'erreur dans le prénom, la bonne valeur est 'Jacques'.
2. Afficher la liste des clés du dictionnaire.
3. Afficher la liste des valeurs du dictionnaire.
4. Afficher la liste des paires clé/valeur du dictionnaire.
5. Ecrire la phrase "Jacques Dupuis a 30 ans".

```
In [19]: d=dict([('nom','Dupuis'),('prenom','Jacque'),('age',30)])

In [20]: d
Out[20]: {'nom': 'Dupuis', 'prenom': 'Jacque', 'age': 30}

In [21]: d['prenom']='Jacques'

In [22]: d
Out[22]: {'nom': 'Dupuis', 'prenom': 'Jacques', 'age': 30}

In [23]: for cle in d.keys(): print(cle)
nom
prenom
age

In [24]: for val in d.values(): print(val)
Dupuis
Jacques
30

In [25]: for cle, val in d.items(): print(cle, val)
nom Dupuis
prenom Jacques
age 30

In [27]: print(d['prenom'], d['nom'], 'a', d['age'], 'ans')
Jacques Dupuis a 30 ans
```

Les conteneurs :Résumé

- séquences ordonnées, accès index rapide, valeurs répétables

```
  list  [1,5,9]   ["x",11,8.9]  ["mot"]     []
 tuple  (1,5,9)    11,"y",7.4   ("mot",)    ()
```

non modifiable

`str` en tant que séquence ordonnée de caractères

expression juste avec des virgules

- sans ordre *a priori*, clé unique, accès par clé rapide ; clés = types de base ou tuples

```
  dict  {"clé":"valeur"}                          {}
```

dictionnaire
couples clé/valeur

```
        {1:"un",3:"trois",2:"deux",3.14:"π"}
```

ensemble

```
   set  {"clé1","clé2"}    {1,9,3,0}     set()
```